

lqcddb 包物理逻辑与代码逻辑解析

——格点 QCD 蒸馏关联函数计算与统计分析工具包

基于 `/root/lattice-pdf/examples/sush/lqcddb/` 源码分析

参考: `/root/lattice-pdf/docs/` 与 `/root/lattice-pdf/books/`

2026 年 7 月 31 日

目录

1 物理基础：格点 QCD 中的蒸馏方法	2
1.1 欧几里得关联函数	2
1.2 蒸馏 (Distillation) 方法	2
1.3 Wick 收缩与费米子符号	3
1.4 顶角函数: VVV 与 VDV	3
1.5 有效质量与 Jackknife 分析	3
2 lqcddb 代码架构	4
2.1 包结构与懒加载机制	4
2.2 全局后端切换	5
2.3 子包功能矩阵	5
3 核心算法一：Wick 收缩自动生成	6
3.1 算符 DSL	6
3.2 Wick 收缩算法流程	6
3.3 等价图识别	7
3.4 指标字符串与自由指标提取	8
4 核心算法二：动态收缩系统	9
4.1 注册表设计	9
4.2 <code>dynamic_contraction</code> 类	10
4.3 双侧投影 (Baryon Projection)	10
4.4 收缩路径优化与缓存	12
5 核心算法三：本征矢量与顶点函数	13
5.1 本征矢量代数 (<code>vector_creator</code>)	13
5.2 动量相因子	14
5.3 VDV 顶点 (带规范链接)	15

目录	2
5.4 VVV 顶点	16
5.5 Ω 蒸馏权重矩阵	16
6 核心算法四：统计分析	17
6.1 Jackknife 重采样	17
6.2 有效质量提取	18
6.3 三点函数比值 (ratio_3pt)	19
6.4 GEVP 求解	20
6.5 非连通图分析 (dis_connect)	20
7 运行实例分析	20
7.1 实例一：质子-中子两点函数 (contraction.pn.2pt.cupy.py)	20
7.2 实例二：质子-流-中子三点函数 (contraction.PJN.3pt.cupy.py)	22
7.3 实例三：动态收缩（实验性 API）	23
7.4 实例四：统计分析 workflow	24
8 代码逻辑与物理逻辑的对应关系	24
9 矩阵体系 (DeGrand-Rossi 基)	24
10 张量指标约定	25
11 关键设计模式	25
11.1 懒加载与依赖隔离	25
11.2 注册表模式 (Registry Pattern)	25
11.3 计划缓存 (Plan Cache)	25
11.4 cached_contract 缓存策略	25
12 数据流全景	25

1 物理基础：格点 QCD 中的蒸馏方法

1.1 欧几里得关联函数

格点 QCD 在四维欧几里得时空中计算强子关联函数。对于强子算符 $O(\vec{x}, t)$ ，零动量投影的两点关联函数为：

两点关联函数定义

$$C_{2\text{pt}}(t) = \sum_{\vec{x}} \langle \Omega | O(\vec{x}, t) O^\dagger(\vec{0}, 0) | \Omega \rangle = \langle \Omega | O(t) O^\dagger(0) | \Omega \rangle \quad (1)$$

在大时间分离极限下， $C_{2\text{pt}}(t) \propto e^{-E_0 t}$ ，其中 E_0 为基态能量。

类似地，三点关联函数涉及流算符 $J(\vec{x}, \tau)$ 的插入：

三点关联函数

$$C_{3\text{pt}}(t_{\text{sep}}, \tau) = \sum_{\vec{x}, \vec{y}} \langle \Omega | O_f(\vec{x}, t_{\text{sep}}) J(\vec{y}, \tau) O_i^\dagger(\vec{0}, 0) | \Omega \rangle \quad (2)$$

其中 t_{sep} 为源-汇分离， τ 为流插入时间。在大时间极限下提取矩阵元 $\langle f | J | i \rangle$ 。

1.2 蒸馏 (Distillation) 方法

蒸馏方法 [?] 的核心思想是用拉普拉斯算符的低模本征矢量构造一个垂直于高能模式的投影算符。对于三维空间拉普拉斯算符：

蒸馏投影算符

$$-\nabla_{xy}^2(t) = 6\delta_{xy} - \sum_{j=1}^3 \left(U_j(x, t) \delta_{x+\hat{j}, y} + U_j^\dagger(x - \hat{j}, t) \delta_{x-\hat{j}, y} \right) \quad (3)$$

求解本征值问题 $-\nabla^2(t) \xi^{(k)}(t) = \lambda^{(k)}(t) \xi^{(k)}(t)$ ，取前 N_{ev} 个本征矢量构造投影算符：

$$\square_{xy}(t) = \sum_{k=1}^{N_{\text{ev}}} \xi_x^{(k)}(t) \xi_y^{(k)\dagger}(t) \quad (4)$$

蒸馏后的夸克传播子 (perambulator) 为：

$$\tau_{\alpha\beta, kl}^{(t_{\text{sink}}, t_{\text{src}})} = \xi^{(k)\dagger}(t_{\text{sink}}) D_{\alpha\beta}^{-1}(t_{\text{sink}}, t_{\text{src}}) \xi^{(l)}(t_{\text{src}}) \quad (5)$$

其中 D^{-1} 为 Dirac 算符的逆 (夸克传播子)， α, β 为 Dirac 指标， k, l 为本征矢量指标。

蒸馏方法的优势在于：

1. **维度压缩**：将 $12V \times 12V$ 的 Dirac 矩阵压缩为 $4N_{\text{ev}} \times 4N_{\text{ev}}$ 的 perambulator 矩阵
2. **算符构造灵活**：任意强子插值算符可事后构造，无需重新求逆
3. **动量投影高效**：通过本征矢量的相因子投影实现任意动量的关联函数

1.3 Wick 收缩与费米子符号

对于给定的初态和末态强子算符，关联函数的计算归结为对所有可能的 Wick 收缩求和。每个收缩图对应一个特定的费米子配对方式，其振幅由 perambulator、顶角函数和 γ 矩阵的张量收缩给出。

费米子反对称性导致每个 Wick 收缩带有一个符号 $(-1)^P$ ，其中 P 为将夸克场排列成收缩对所需的置换奇偶性。

Wick 定理应用于强子关联函数

对于重子算符（如质子 $O = \epsilon^{abc}(u^a C \gamma_5 d^b) u^c$ ），关联函数展开为：

$$\langle O(t) O^\dagger(0) \rangle = \sum_{\sigma \in S_3 \times S_3} \text{sgn}(\sigma) \prod_f \text{Tr}[\Gamma_\sigma \tau_f \Gamma'_\sigma \tau_f^\dagger] \quad (6)$$

其中 S_3 为三夸克置换群， Γ 表示 γ 矩阵结构。

1.4 顶角函数：VVV 与 VDV

在蒸馏框架中，强子算符中的色指标收缩通过本征矢量的组合实现：

三夸克顶角 VVV

对于重子算符 $\epsilon^{abc} q^a q^b q^c$ ，三夸克顶角为：

$$\text{VVV}_{klm}(\vec{p}, t) = \epsilon^{abc} \sum_{\vec{x}} e^{-i\vec{p} \cdot \vec{x}} \xi_{\vec{x}}^{(k)a}(t) \xi_{\vec{x}}^{(l)b}(t) \xi_{\vec{x}}^{(m)c}(t) \quad (7)$$

夸克-反夸克顶角 VDV

对于介子算符 $\bar{q} \Gamma q$ （或重子中的双夸克对），VDV 顶角为：

$$\text{VDV}_{kl}(\vec{p}, t) = \sum_{\vec{x}} e^{-i\vec{p} \cdot \vec{x}} \xi_{\vec{x}}^{(k)\dagger a}(t) \xi_{\vec{x}}^{(l)a}(t) \quad (8)$$

可以扩展为包含规范链接的版本：

$$\text{VDV}_{kl}(\vec{p}, \Delta\vec{x}, t) = \sum_{\vec{x}} e^{-i\vec{p} \cdot \vec{x}} \xi_{\vec{x}}^{(k)\dagger a}(t) U^{ab}(\vec{x}, \vec{x} + \Delta\vec{x}) \xi_{\vec{x} + \Delta\vec{x}}^{(l)b}(t) \quad (9)$$

1.5 有效质量与 Jackknife 分析

从两点关联函数提取能谱的标准方法：

有效质量

$$E_{\text{eff}}^{\text{log}}(t) = \frac{1}{a} \ln \left[\frac{C_{2\text{pt}}(t)}{C_{2\text{pt}}(t+1)} \right] \quad (10)$$

$$E_{\text{eff}}^{\text{cosh}}(t) = \frac{1}{a} \text{arccosh} \left[\frac{C_{2\text{pt}}(t+2) + C_{2\text{pt}}(t)}{2C_{2\text{pt}}(t+1)} \right] \quad (11)$$

其中 a 为格距, $E_{\text{eff}}(t) \rightarrow E_0$ 当 $t \rightarrow \infty$ (基态平台区)。

Jackknife 重采样用于估计统计误差。对 N 个组态, 第 k 个 Jackknife 样本是去掉第 k 个组态后其余 $N-1$ 个组态的均值:

$$\bar{x}_{(k)} = \frac{1}{N-1} \sum_{i \neq k} x_i, \quad \sigma_{\text{jk}} = \sqrt{\frac{N-1}{N} \sum_{k=1}^N (\bar{x}_{(k)} - \bar{x})^2} \quad (12)$$

2 lqcddb 代码架构

2.1 包结构与懒加载机制

lqcddb 采用 Python 包标准布局, 通过 `pyproject.toml` 管理依赖。其核心特色是 `__init__.py` 中的懒加载 (lazy-loading) 机制:

`__init__.py` 懒加载核心逻辑

```

1 _ATTR_TO_MODULE: dict[str, str] = {}
2
3 # 将每个公开名映射到具体源模块
4 for _name in ("wick_contraction", "plot_figure_wick", ...):
5     _ATTR_TO_MODULE[_name] = ".contraction.autowick"
6
7 for _name in ("Jackknife", "Bootstrap", "meff", "ratio_3pt", ...):
8     _ATTR_TO_MODULE[_name] = ".analyse.analyse"
9
10 def __getattr__(name: str):
11     if name in _ATTR_TO_MODULE:
12         mod_path = _ATTR_TO_MODULE[name]
13         if mod_path not in _MODULES:
14             from importlib import import_module
15             _MODULES[mod_path] = import_module(mod_path, __name__)
16             return getattr(_MODULES[mod_path], name)
17         raise AttributeError(...)

```

设计优势: `from lqcddb import *` 不加载任何子包——仅在首次访问属性时才导入对应的源模块。这避免了加载 `mpi4py`、`cupy`、`sympy` 等重依赖, 除非实际使用。

2.2 全局后端切换

所有数组操作通过模块级全局状态路由到 `numpy` 或 `cupy`:

后端切换核心代码 (base/backend.py)

```

1 _BACKEND = None
2
3 def set_backend(backend: Literal["numpy", "cupy"]):
4     global _BACKEND
5     if backend == "numpy":
6         import numpy
7         _BACKEND = numpy
8     elif backend == "cupy":
9         import cupy
10        _BACKEND = cupy
11
12 def get_backend():
13     if _BACKEND is None:
14         set_backend("numpy")
15     return _BACKEND

```

注意事项: 后端状态是全局且非线程安全的。需要临时切换后端的函数必须保存并恢复原后端。

2.3 子包功能矩阵

子包	核心导出	功能
base/	set_backend, get_backend, cached_contract, levi_civita_tensor, ArraySlicer, SU2combine, mpinit, stout_smear_ndarray	后端切换、缓存 einsum、Levi-Civita、SU(2) CG 系数、MPI 初始化、规范场平滑
constant/	Nc, Ns, Nd, fm2GeV, gamma(), sigma()	物理常量、18 种 DR 基 γ 矩阵
contraction/	wick_contraction, dynamic_contraction, PeramRegistry, VRegistry, GammaRegistry, seq_peram, conjugate_operator, analyze_bandwidth	Wick 收缩引擎、注册表系统、等价图识别、 γ_5 时序变换、算符共轭、Roofline 带宽分析
eigvectors/	vector_creator, vertex_creator	本征矢量代数、VdV/VVV 顶点
analyse/	Jackknife, Bootstrap, meff, ratio_3pt, solve_gevp, loop_tsrc, dis_connect	统计重采样、有效质量、GEVP 求解、源平均、非连通图
io/	write_data_ascii	二进制/ASCII 本征矢读写

表 1: lqcddb 子包功能矩阵

3 核心算法一：Wick 收缩自动生成

3.1 算符 DSL

wick_contraction 使用字符串列表定义强子算符：

算符格式约定

```

1 # 味标记
2 'u', 'd', 's', 'c', 'b', 't'          # 夸克
3 'u^d', 'd^d', 's^d'                  # 反夸克 (^d = dagger)
4
5 # 通配符 (自动展开)
6 'q', 'q^d'                            # 全部六味
7 'l', 'l^d'                            # 轻味 (u, d)
8
9 # 矩阵插入
10 'gamma_5', 'gamma_mu', 'gamma_4', 'gamma_7'
11
12 # 强子分隔符与系数
13 '|'                                    # 强子边界
14 1, -1, 2.0                            # 数值系数
15
16 # 实例
17 pion = ['u^d', 'gamma_5', 'd']         # pi+
18 proton = ['u', 'u', 'C*gamma_5', 'd']  # 质子
19 neutron = ['d', 'u', 'C*gamma_5', 'd'] # 中子

```

3.2 Wick 收缩算法流程

算法 3.1 (Wick 收缩主流程). 设 $\{q_i\}$ 为夸克场集合, $\{\bar{q}_j\}$ 为反夸克场集合。算法流程如下：

1. **味分组**：将夸克和反夸克按味分类。对每味 f ，必须有 $N_q^f = N_{\bar{q}}^f$ （否则该味替换无效）。
2. **生成合法配对**：对每味 f ，枚举所有完美匹配 (*perfect matching*)。对于 n 对夸克-反夸克，共有 $n!$ 种配对方式。
3. **笛卡尔积组合**：取各味配对的笛卡尔积，得到全部 Wick 图。
4. **计算费米子符号**：对每个图，将所有场按算符字符串中的位置排序，计算置换逆序数。符号 $= (-1)^{\#inversions}$ 。
5. **生成指标字符串**：为每个 $peram$ 分配收缩指标 (小写字母)，为每个 γ 矩阵和 V 顶点分配组合指标 (大写字母前缀 + 相邻夸克指标)。
6. **通配符展开**：若含有 'q' 或 'l'，对每种味替换生成独立的 *dict*。

费米子符号计算

```
1 def compute_fermion_sign(contraction_pairs: list) -> int:
2     indexed_positions = []
3     for pair_idx, pair in enumerate(contraction_pairs):
4         quark_op_pos = pair[0]          # 夸克在算符中的位置
5         antiq_op_pos = pair[1]         # 反夸克在算符中的位置
6         indexed_positions.append((quark_op_pos, pair_idx))
7         indexed_positions.append((antiq_op_pos, pair_idx))
8
9     pair_indices = [x[0] for x in indexed_positions]
10    return (-1) ** count_inversions(pair_indices)
```

3.3 等价图识别

identify_equivalent_diagrams 通过比较 peram 配对结构识别等价的 Wick 图:

等价图识别算法

```

1 def identify_equivalent_diagrams(*dicts):
2     # 对每个图的 peram 列表, 生成所有 转置变体
3     for dict_idx, _dicts in enumerate(dicts):
4         gamma_names = [x[1] for x in _dicts['gamma_pos']]
5         change_quark_pos = [[int(x[0])-1, int(x[0])+1]
6                             for x in _dicts['gamma_pos']]
7
8         for diag_idx, peram_entry in enumerate(peram):
9             variants = generate_all_swaps(peram_entry,
10                                           change_quark_pos)
11             sorted_variants = [sorted(v, key=lambda x: x[0])
12                               for v in variants]
13             data.append(sorted_variants)
14
15     # Union-Find 聚类: 共享任意变体的图属于同一等价类
16     raw_groups = find_connected_groups(data)
17
18     # BFS 传播相对符号
19     for group in raw_groups:
20         # 从 canonical 图出发, 沿变体共享边传播系数
21         coeffs = {canonical_idx: canonical_rs}
22         queue = deque([canonical_idx])
23         while queue:
24             cur = queue.popleft()
25             # 检查当前图的所有变体...
26             # 通过变体共享边计算邻居的系数
27             n_coeff = cur_coeff * (n_rs / cur_rs) * (
28                 n_vs_list[v_neighbor] / cur_vs_list[v_cur])
29             coeffs[neighbor] = n_coeff

```

核心思想: 两张图若通过 γ 矩阵两侧的夸克交换可以互相转化, 则它们是等价的。算法使用并查集 (Union-Find) 对图进行聚类, 然后用 BFS 沿变体共享边传播相对系数。

3.4 指标字符串与自由指标提取

Wick 收缩的输出包含 einsum 指标字符串, 格式为:

$$\text{peram_indices, gamma_indices, V_indices} \rightarrow \text{free_indices} \quad (13)$$

其中 `keep_unique_letters` 函数从收缩指标中提取自由指标 (小写字母只保留出现一次的, 大写字母保留首次出现的), 作为输出张量的轴标签。

4 核心算法二：动态收缩系统

4.1 注册表设计

dynamic_contraction 系统通过三个注册表类解耦数据加载与收缩计算：

PeramRegistry —Perambulator 注册表

```

1 class PeramRegistry:
2     def __init__(self):
3         self._entries = {} # {(flavor, (t_q, t_aq)): data}
4
5     def register(self, flavor, time_labels, data):
6         # flavor: 'u', 'd', 's', 'light'
7         # time_labels: ('tsink', 'tsrc'), ('tsink', 'tcur0'), ...
8         # data: ndarray, shape (... , Ns, Ns, Nev, Nev)
9         self._entries[(flavor, tuple(time_labels))] = data
10
11     def resolve(self, combined_type, time_labels):
12         qf = combined_type[0] # 从 'uu~d' 提取夸克味 'u'
13         key = (qf, tuple(time_labels))
14         if key in self._entries:
15             return self._entries[key]
16         # 'light' 通配 u, d
17         key_light = ('light', tuple(time_labels))
18         if key_light in self._entries:
19             return self._entries[key_light]
20         raise KeyError(...)

```

关键设计决策：注册表存储引用而非复制——调用者管理数组生命周期，传递视图（切片）以减少内存。

时间标签命名规范

标签	含义
'tsink'	汇端 (sink) 时间片
'tsrc'	源端 (source) 时间片
'tcur0'	第 0 个流插入点 (3pt)
'tcur1', 'tcur2', ...	多点函数 (4pt+) 的额外插入点

4.2 dynamic_contraction 类

dynamic_contraction 初始化流程

```

1 class dynamic_contraction:
2     def __init__(self, operator_groups, *, peram_registry,
3                 v_registry, gamma_registry, Cpt='2pt', ...):
4         # Step 1: 运行 Wick 分析 (带缓存)
5         self.plan = run_wick_analysis(operator_groups, Cpt=Cpt, ...)
6
7         # Step 2: 校验注册完整性
8         self.missing = []
9         for entry in self.plan:
10             # 检查每个 peram/V/gamma 是否已注册
11             w = entry[2]; di = entry[3]
12             for p in w['peram'][di]:
13                 try: peram_registry.resolve(p[2], p[4])
14                 except KeyError: self.missing.append(...)
15             # ... V 和 gamma 的类似检查
16
17     def calculate(self, index):
18         entry = self.plan[index]
19         # 等价类系数之和为 0 → 跳过
20         coeffs = [c for _, _, c in entry[0]]
21         total_coeff = sum(coeffs)
22         if abs(total_coeff) < 1e-12: return 0
23
24         # 解析张量 + cached_contract
25         return calculate_contraction(entry, ...)
26
27     def calculate_all(self):
28         result = 0
29         for i in range(len(self.plan)):
30             result = result + self.calculate(i)
31         return result

```

4.3 双侧投影 (Baryon Projection)

重子关联函数需要将自旋指标通过投影算符收缩。dynamic_contraction 支持通过 GammaRegistry 注册 'Projector' 键 (两个投影算符的序列 [proj_sink, proj_src]) 来实现:

双侧投影 einsum 构建

```

1 def _build_projection_einsum(ein, proj_label, output_labels,
2                               oindex_given=False):
3     lhs, rhs = ein.split('->')
4     spin_out = rhs[:2] # 两个输出自旋指标
5     a, s = spin_out[0], spin_out[1]
6
7     if oindex_given:
8         # 投影指标互相收缩: 输出严格按 Oindex
9         z = [c for c in 'ABC...XYZ' if c not in used][0]
10        return (f'{lhs},{proj_label}{a}{z},{proj_label}{s}{z}'
11                f'->{output_labels}')
12
13    # 默认: 引入两个新自由指标 X, Y
14    x, y = [c for c in 'ABC...XYZ' if c not in used][:2]
15    return (f'{lhs},{proj_label}{a}{x},{proj_label}{s}{y}'
16            f'->{x}{y}{proj_label}{output_labels}')

```

例如, 质子投影算符 $\mathcal{P} = \frac{1}{2}(\gamma_0 + \gamma_4)$ 在代码中通过 `projection = (gamma(0) + gamma(4))/2` 注册。

4.4 收缩路径优化与缓存

cached_contract — 带缓存的张量收缩

```

1 _expr_cache: Dict[Tuple, Any] = {}
2
3 def cached_contract(einsum_str, *tensors, optimize='auto'):
4     shapes = tuple(t.shape for t in tensors)
5     key = (einsum_str, shapes, opt_key)
6
7     # 缓存命中 → 直接执行
8     expr = _expr_cache.get(key)
9     if expr is not None:
10         return expr(*tensors)
11
12     # 缓存未命中 → 编译路径 + 择优
13     placeholders = [np.broadcast_to(np.empty(()), dtype=t.dtype), t.
14                     shape)
15                     for t in tensors]
16
17     best_path, best_cost = None, (float('inf'), float('inf'))
18     for opt in candidate_opts: # auto, greedy, optimal, dp
19         path, path_info = contract_path(einsum_str, *placeholders,
20                                         optimize=opt)
21         cost = (path_info.opt_cost, path_info.largest_intermediate)
22         if cost < best_cost:
23             best_cost, best_path = cost, path
24
25     expr = contract_expression(einsum_str, *shapes, optimize=best_path)
26     _expr_cache[key] = expr
27     return expr(*tensors)

```

关键点：首次调用时尝试 auto、greedy、optimal、dp 四种策略，选择 FLOP 最小的路径；后续调用直接复用编译后的表达式。

5 核心算法三：本征矢量与顶点函数

5.1 本征矢量代数 (vector_creator)

本征矢量正交归一化检查

```

1 class vector_creator:
2     def check(self, eigvecs, dtype='find', tol=1e-10,
3               check_normal=True):
4         # 展开空间+色指标
5         V = np.prod(eigvecs.shape[1:])
6         eigvecs_flat = eigvecs.reshape(eigvecs.shape[0], V)
7
8         # 计算重叠矩阵  $A_{\{kn\}} = \langle x_{i_k} | x_{i_n} \rangle$ 
9         A = contract('nV,NV->nN', eigvecs_flat,
10                      eigvecs_flat.conj())
11
12         # 检查归一化:  $|A_{\{ii\}} - 1| < tol$ 
13         if check_normal:
14             for i in range(eigvecs.shape[0]):
15                 if ((A[i,i] - 1) * (A[i,i] - 1).conj()).real >= tol:
16                     return False # 不正交
17                 A[i,i] -= 1
18
19         # 检查正交性:  $|A_{\{i j\}}| < tol$ 
20         return not (A >= tol).any()

```

四种压缩方法 (compress_matrix_V1-V4):

- **V1 (Sum)**: 将每组 $N_{\text{eigen}}/N_{\text{sum}}$ 个本征矢求和为一个
- **V2 (Extract)**: 从每组中随机抽取 N_{extract} 个
- **V3 (Random+Orth)**: 随机线性组合后正交归一化
- **V4 (General)**: 支持正交归一化或 $\mathbb{Z}_N$ 随机向量

压缩策略支持三种分组模式:

- **I (Interlace)**: 交错式——将所有本征矢均匀交错分配到各组
- **B (Block)**: 分块式——连续的本征矢分入同一组
- **BI (Block-Interlace)**: 第一维分块, 第二维交错

5.2 动量相因子

相因子计算 (phase_exp_2pt)

```

1 def phase_exp_2pt(self, Mom=[0,0,0]):
2     if all(x == 0 for x in Mom):
3         return backend.ones((Nx, Nx, Nx, Nc), dtype=complex)
4
5     factor = -2j * pi / Nx
6     z_phase = exp(factor * Mom[0] * z[:, None, None])
7     y_phase = exp(factor * Mom[1] * y[None, :, None])
8     x_phase = exp(factor * Mom[2] * x[None, None, :])
9
10    phase_3d = z_phase * y_phase * x_phase # 广播组合
11    return stack([phase_3d, phase_3d, phase_3d], axis=-1)

```

数学上，动量投影相因子为：

$$e^{-i\vec{p}\cdot\vec{x}} = \exp\left(-\frac{2\pi i}{L}(p_z z + p_y y + p_x x)\right) \quad (14)$$

注意动量顺序为 $[p_z, p_y, p_x]$ (z 分量最快)。

5.3 VDV 顶点 (带规范链接)

VdV_sink_t_link — 三种计算情形

```

1 def VdV_sink_t_link(self, eigvecs, phase_exp, link_dir,
2                       link_max, gauge_link, eigvecs_max, conserved):
3     # 情形1: 无规范链接
4     if gauge_link is None or isinstance(gauge_link, bool):
5         #  $V_{\{mn\}}(p) = \sum_x e^{-ipx} x_{i_m}(x) x_{i_n}(x)$ 
6         VDV[:,0,[:,:] = contract('VN,mV,nV->mNn',
7                                   eigvecs_conj_T, phase_exp, eigvecs_flat)
8
9     # 情形2: 守恒流 / 时间方向
10    if conserved or link_dir == 'T':
11        #  $V[0] = x_i * @ U_t @ x_{i_{max}}$ 
12        #  $V[1] = x_{i_{max}} * @ U_t * @ x_i$ 
13        VDV[0] = contract('nvc,vcb,Nvb->nN',
14                          ev.conj(), glink, ev_max)
15        VDV[1] = contract('nvc,vbc,Nvb->nN',
16                          ev_max.conj(), glink.conj(), ev)
17
18    # 情形3: 空间方向 (X/Y/Z/all)
19    for link_indx in range(-link_max, link_max+1):
20        eig_rolled = roll(eigvecs, -link_indx, axis=roll_ax)
21        # 构建规范输运路径
22        if link_indx != 0:
23            gauge_path = 构建 U(x)...U(x+link_indx-1)
24            link_rolled = contract('vcb,Nvb->Nvc',
25                                  gauge_path, eig_rolled)
26        # 对所有动量同时收缩
27        VDV[:, link_indx+link_max] += contract(
28            'VN,mV,nV->mNn', eigvecs_conj_T, phase_exp, link_flat)

```

5.4 VVV 顶点

Mom_VVV_sink_t —三夸克顶角

```

1 def Mom_VVV_sink_t(self, phase_exp, eigvecs):
2     VVV = zeros((num_Mom, Nev, Nev, Nev), dtype=complex)
3
4     for dir in range(1, Nx+1):
5         # 六个排列项：完全反对称化
6         VVV += contract("Mzyx,azyx,bzyx,czyx->Mabc",
7             phase_exp, eig_a, eig_b, eig_c)      # +abc
8         VVV += contract("Mzyx,azyx,bzyx,czyx->Mabc",
9             phase_exp, eig_b, eig_c, eig_a)      # +bca
10        VVV += contract("Mzyx,azyx,bzyx,czyx->Mabc",
11            phase_exp, eig_c, eig_a, eig_b)      # +cab
12        VVV -= contract("Mzyx,azyx,bzyx,czyx->Mabc",
13            phase_exp, eig_a, eig_c, eig_b)      # -acb
14        VVV -= contract("Mzyx,azyx,bzyx,czyx->Mabc",
15            phase_exp, eig_b, eig_a, eig_c)      # -bac
16        VVV -= contract("Mzyx,azyx,bzyx,czyx->Mabc",
17            phase_exp, eig_c, eig_b, eig_a)      # -cba
18
19    return VVV

```

数学上，VVV 顶角编码了 ϵ^{abc} 颜色反对称化 + 颜色空间本征矢量的三线组合：

$$VVV_{klm}(\vec{p}) = \sum_{\vec{x}} e^{-i\vec{p}\cdot\vec{x}} \epsilon^{abc} \xi_{\vec{x}}^{(k)a} \xi_{\vec{x}}^{(l)b} \xi_{\vec{x}}^{(m)c} \quad (15)$$

六个排列项对应 ϵ^{abc} 的六个非零分量 (+1 或 -1)。

5.5 Ω 蒸馏权重矩阵

`create_omega_accelerate` 构建任意维度的蒸馏权重张量：

$$\Omega_{i_1 i_2 \dots i_D} = \prod_{d=1}^D \sqrt{\frac{N_{\text{space}}^{(i_d)} - c_d}{N_{\text{sum}}^{(i_d)} - c_d}} \quad (16)$$

其中 c_d 为维度 d 上已使用的索引计数（处理对角元素的权重衰减）。

6 核心算法四：统计分析

6.1 Jackknife 重采样

Jackknife 实现

```

1 def Jackknife(data, Nconf_axes=0, cov_axes=None):
2     Nconf = data.shape[Nconf_axes]
3
4     # 1. Jackknife 样本:  $x_{\{k\}} = -(x - \text{sum}(x)) / (N-1)$ 
5     data_sum = sum(data, axis=Nconf_axes, keepdims=True)
6     data_sample = -(data - data_sum) / (Nconf - 1)
7
8     # 2. 均值与误差
9     data_mean = mean(data, axis=Nconf_axes)
10    data_err = sqrt(Nconf - 1) * std(data_sample, axis=Nconf_axes)
11
12    # 3. 协方差矩阵 (沿指定轴)
13    if cov_axes is not None:
14        residual = data_sample - data_mean
15        cov = einsum('n...i,n...j->...ij', r_flat, r_flat)
16        cov *= (Nconf - 1) / Nconf
17
18    return {'data_sample': data_sample, 'data_mean': data_mean,
19            'data_err': data_err, 'data_cov': cov}

```

6.2 有效质量提取

meff — 三种有效质量方法

```
1 def meff(data_sample, alttc, Nconf_axes=0, Nt_axes=1,
2         meff_type='log'):
3     if meff_type == 'log':
4         meff[t] = log(C(t) / C(t+1)) * (fm2GeV / alttc)
5
6     elif meff_type == 'cosh':
7         meff[t] = arccosh((C(t+2) + C(t)) / (2*C(t+1)))
8                 * (fm2GeV / alttc)
9
10    elif meff_type == 'GEVP':
11        meff[t] = log(lambda(t) / lambda(t+1)) * (fm2GeV / alttc)
12
13    # 对 Jackknife 样本取统计
14    meff_mean = mean(meff_sample, axis=Nconf_axes)
15    meff_err = std(meff_sample, axis=Nconf_axes) * sqrt(Nconf-1)
16    return {'data_sample': meff_sample, 'data_mean': meff_mean,
17            'data_err': meff_err}
```

6.3 三点函数比值 (ratio_3pt)

ratio_3pt —PDF 矩阵元提取

```

1 def ratio_3pt(data_3pt_sample, data_2ptI_sample,
2               data_2ptF_sample=None, t_sep=12, ...):
3     # 公式:
4     #  $R = C3 / C2F(tsep) * \sqrt{C2I(tsep-\tau) C2F(\tau) C2F(tsep) / (C2F(tsep-\tau) C2I(\tau) C2I(tsep))}$ 
5     #
6     # )
7
8
9     # Link折叠 (前向+后向平均)
10    if link_fold and link_axes is not None:
11        data_3pt_sample = _fold_link(data_3pt_sample, link_axes)
12
13    # 1D模式: 单时间轴
14    if t_src_axes is None:
15        C2F_tsep_minus_tau = slice(C2F, t_sep - arange(Ntau))
16        sqrt_term = sqrt(max(
17            C2I(tshift)*C2F(tau)*C2F(tsep) /
18            (C2F(tshift)*C2I(tau)*C2I(tsep)), 0))
19        ratio_sample = C3 / C2F(tsep) * sqrt_term
20
21    # 2D模式: 双时间轴 (源+tau)
22    else:
23        for t_src in range(Nt_src):
24            # 循环源时间, 计算每个tsrc的比值
25            ratio_slice = C3_by_tau / C2F_tsep * sqrt_term
26
27    ratio_mean = mean(ratio_sample.real, axis=Nconf_axes)
28    ratio_err = std(ratio_sample.real, axis=Nconf_axes)*sqrt(Nconf-1)

```

6.4 GEVP 求解

solve_gevp — 广义特征值问题

```

1 def solve_gevp(C, t0):
2     # C: (N, N, Nt) 关联函数矩阵
3     for t in range(Nt):
4         # 求解  $C(t) v = \lambda C(t_0) v$ 
5         eigenvalues, eigenvectors = eigh(C[...,t], C[...,t0])
6
7         if t < t0:
8             order = argsort(eigenvalues)          # 升序
9         else:
10            order = argsort(eigenvalues)[::-1]      # 降序
11
12            eigenvalues = eigenvalues[order]
13            # 归一化特征矢量:  $v \rightarrow v / \sqrt{v^* @ v}$ 
14            eigenvectors = eigenvectors[:, order]
15            coeff = eigenvectors.conj().T @ eigenvectors
16            eigenvectors = eigenvectors / sqrt(diag(coeff))
17
18     return C_GEVP, C_eigenvectors

```

6.5 非连通图分析 (dis_connect)

dis_connect 实现非连通图贡献的计算: $C_{3pt}^{disc} = C_{2pt}^{\mu\nu} - \langle C_{2pt}^{\mu\nu} \rangle \langle C_{bubble} \rangle$, 用于胶子 PDF 的 OPE 部分。

7 运行实例分析

7.1 实例一: 质子-中子两点函数 (contraction.pn.2pt.cupy.py)

这是一个典型的 GPU 蒸馏收缩计算。完整流程:

实例一主要流程

```

1 # 初始化
2 set_backend('cupy')
3 mpinit(grid_size=[1,1,1,1], latt_size=[32,32,32,64])
4
5 # 动量列表生成
6 Mom_sink_VdV = [[0,0,0]] + creat_mom_list([0,0,1])
7               + creat_mom_list([0,1,1]) + creat_mom_list([1,1,1])
8 Mom_sink_VVV = [[0,0,0]] + ... # 反序
9
10 # 加载本征矢量 + 计算 VdV/VVV
11 for t_src in t_rank:
12     eigvecs = load(f'{eigen_path}/{conf_id}_t{t_src:03d}_...np')
13     sink_VdV[t_src_indx] = Mom_VdV_sink_t(phase_exp, eigvecs)
14     sink_VVV[t_src_indx] = Mom_VVV_sink_t(phase_exp, eigvecs)
15
16 # 主循环：对每个源时间片
17 for t_src in range(Nt):
18     # 加载 perambulator + seq_peram 获取反夸克传播子
19     peram_u = load(f'{peram_path}/t{t_src:03d}_...np')
20     peram_d = seq_peram(peram_u) # gamma_5 时序变换
21
22     for t_sink in t_sink_list:
23         # 12个 Wick 收缩图 (中子 pi+道)
24         corr[0] = -contract('oapb,kelf,...,ek->NM',
25                             peram_u, peram_u, ..., gamma(7), gamma(5), ...,
26                             source_VVV, source_VdV, sink_VdV, sink_VVV, projection)
27         # ... 11 more diagrams ...
28
29 # 源平均 + 保存
30 corr = loop_tsrc(corr, indx=[-2,-1],
31                 Boundary_Conditions='Antiperiodic')
32 np.save(f'{save_path}/corr_ud_2pt_...np', corr)

```

物理对应关系：

每个 contract 调用对应一个 Wick 收缩图。例如第一个图：

```

contract('oapb,kelf,sgth,icjd,qmrn,ac,mo,gi,qs,Nbdf,Nnp,Mhj,Mlrt,ek->NM',
        peram_u[t_src], peram_u[t_sink], peram_u[t_src], peram_u[t_sink],
        peram_u[t_sink], gamma(7), gamma(5), gamma(5), gamma(7),
        source_VVV, source_VdV, sink_VdV, sink_VVV, projection)

```

指标对应关系：

- oapb, kelf, sgth, icjd, qmrn — 五个 perambulator (5 个夸克传播子 τ)

- `ac`, `mo`, `gi`, `qs` — 四个 γ 矩阵 ($\gamma_7, \gamma_5, \gamma_5, \gamma_7$)
- `Nbdf`, `Nnp`, `Mhj`, `Mlrt` — sink VVV、source VdV、sink VdV、source VVV
- `ek` — 投影算符 $\mathcal{P} = (\gamma_0 + \gamma_4)/2$
- `->NM` — 输出: N 个 sink 动量 $\times M$ 个 source 动量

γ_5 时序变换: `seq_peram(peram)` 实现

$$\tilde{\tau}(t_{\text{sink}}, t_{\text{src}}) = \gamma_5 \tau^*(t_{\text{sink}}, t_{\text{src}}) \gamma_5 \quad (17)$$

将夸克 `peram` 转换为反夸克 `peram` (利用 γ_5 -厄米性)。

7.2 实例二：质子-流-中子三点函数 (contraction.PJN.3pt.cupy.py)

三点函数引入流算符插入, 计算矩阵元 $\langle N | \bar{u} \gamma_\mu d | P \rangle$:

实例二关键差异

```

1 # 构建弱流 gamma_w = (1-gamma_5) * gamma_mu
2 gamma_curr = (gamma(0) - gamma(5)) @ [gamma(1), gamma(2), gamma(3), gamma
   (4)]
3
4 # 对每个流插入时间 t_curr
5 for t_curr in t_curr_list_indx:
6     peram_u_src_seq = seq_peram(peram_u_src[t_curr])
7
8     # 四个 Wick 图求和
9     corr_3pt += contract(
10         'eifj,okpl,ambn,cgdh,ce,Gmo,gi,Nbdf,NLnp,Mhjl,ka->MNGL',
11         peram_u[t_sink], peram_u[t_curr], peram_u_sink_seq[t_curr],
12         peram_u[t_sink], gamma(7), gamma_curr, gamma(7),
13         sink_VVV, VdV_link[t_curr], source_VVV, projection) * (-1.0)
14     # ... +3 more diagrams ...

```

三点函数的物理结构:

输出形状为 (M, N, G, L):

- M — sink 动量 (质子)
- N — source 动量 (中子)
- G — 流算符分量 ($\gamma_1, \gamma_2, \gamma_3, \gamma_4$)
- L — 规范链接位移 ($\Delta z = -L_{\text{max}}, \dots, +L_{\text{max}}$)

7.3 实例三：动态收缩（实验性 API）

test.dynamic.py 演示了 dynamic_contraction 的新 API:

动态收缩 API 实例

```
1 from lqcddb import *
2
3 # 注册数据
4 peram_reg = PeramRegistry()
5 peram_reg.register('light', ('tsink', 'tsrc'), peram_array)
6
7 v_reg = VRegistry()
8 v_reg.register('VVV_0', 'tsink', sink_VVV)
9 v_reg.register('VDV_0', 'tsrc', source_VdV)
10
11 gamma_reg = GammaRegistry()
12 gamma_reg.register('gamma_5', gamma(5))
13 gamma_reg.register('gamma_7', gamma(7))
14
15 # 定义算符
16 sink_op = ['|', 'u', 'u', 'gamma_7', 'd', '|']
17 src_op = ['|', 'u^d', 'gamma_7', 'd^d', 'u^d', '|']
18
19 # 动态收缩
20 dc = dynamic_contraction(
21     [(sink_op, src_op)],
22     peram_registry=peram_reg,
23     v_registry=v_reg,
24     gamma_registry=gamma_reg,
25     Cpt='2pt', use_equivalence=True, ignore_dis=True
26 )
27 result = dc.calculate_all()
```

7.4 实例四：统计分析 workflow

Jackknife + 有效质量

```

1 # 1. 加载多个组态的两点函数
2 corr_2pt = np.stack([np.load(f'conf{c}_2pt.npy')
3                       for c in conf_list]) # (Nconf, Nt)
4
5 # 2. Jackknife重采样
6 jk = Jackknife(corr_2pt, Nconf_axes=0)
7 # jk['data_sample']: (Nconf, Nt)
8 # jk['data_mean']: (Nt,)
9 # jk['data_err']: (Nt,)
10
11 # 3. 有效质量
12 meff_result = meff(jk['data_sample'], alttc=0.1053,
13                    meff_type='cosh')
14 # meff_result['data_mean']: 有效质量均值 vs t
15 # meff_result['data_err']: 有效质量误差
16
17 # 4. 三点函数比值 (PDF矩阵元)
18 ratio = ratio_3pt(data_3pt_sample, data_2ptI_sample,
19                  data_2ptF_sample, t_sep=12,
20                  link_fold=True)
21 # ratio['data_mean']: R(z, tau) 矩阵元

```

8 代码逻辑与物理逻辑的对应关系

9 矩阵体系 (DeGrand-Rossi 基)

DeGrand-Rossi (DR) 基是手征非对角的 γ 矩阵表示:

DR 基 γ 矩阵定义

$$\gamma_0 = I_{4 \times 4}, \quad \gamma_1 = \begin{pmatrix} 0 & 0 & 0 & i \\ 0 & 0 & i & 0 \\ 0 & -i & 0 & 0 \\ -i & 0 & 0 & 0 \end{pmatrix}, \quad \gamma_2 = \begin{pmatrix} 0 & 0 & 0 & -1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 0 \end{pmatrix} \quad (18)$$

$$\gamma_3 = \begin{pmatrix} 0 & 0 & i & 0 \\ 0 & 0 & 0 & -i \\ -i & 0 & 0 & 0 \\ 0 & i & 0 & 0 \end{pmatrix}, \quad \gamma_4 = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}, \quad \gamma_5 = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix} \quad (19)$$

18 种 γ 矩阵组合的物理含义：

在 Wick 收缩中， γ 矩阵插入在夸克传播子之间：

$$[\tau(t_1, t_2) \Gamma \tau(t_2, t_3)]_{\alpha\gamma}^{kl} = \sum_{\beta, m} \tau_{\alpha\beta}^{km}(t_1, t_2) \Gamma_{\beta\gamma} \tau_{\beta\gamma}^{ml}(t_2, t_3) \quad (20)$$

其中 $\Gamma \in \{\gamma_0, \dots, \gamma_{17}\}$ 。

10 张量指标约定

lqcddb 使用的张量形状约定（蒸馏框架标准）：

11 关键设计模式

11.1 懒加载与依赖隔离

`__getattr__ + _ATTR_TO_MODULE` 映射使得重依赖（`mpi4py`, `cupy`, `sympy`）仅在首次访问相关属性时才被导入。这允许在没有 GPU 的环境中安全地 `import lqcddb` 并使用 CPU 功能。

11.2 注册表模式 (Registry Pattern)

`PeramRegistry`、`VRegistry`、`GammaRegistry` 三个注册表类实现数据与计算逻辑的分离：

- 数据按（味，时间标签）或（名称，时间标签）键存储
- 查找时支持通配符（如 'light' 同时匹配 u 和 d）
- 存储引用而非副本——内存由调用者管理

11.3 计划缓存 (Plan Cache)

`run_wick_analysis` 内部维护 `_plan_cache` 字典，键为（算符组，Cpt，指标前缀，等价图选项等）的可哈希元组。相同算符配置的重复调用直接返回缓存结果，避免重新分析 Wick 收缩（计算量随夸克数量阶乘增长）。

11.4 `cached_contract` 缓存策略

`cached_contract` 在模块级维护 `_expr_cache`，键为（`einsum_str`, `shapes`, `opt_key`）。首次调用编译收缩路径（尝试四种优化策略选择 FLOP 最小者），后续调用直接执行编译后的表达式。调用 `clear_cache()` 可清空缓存释放内存。

12 数据流全景

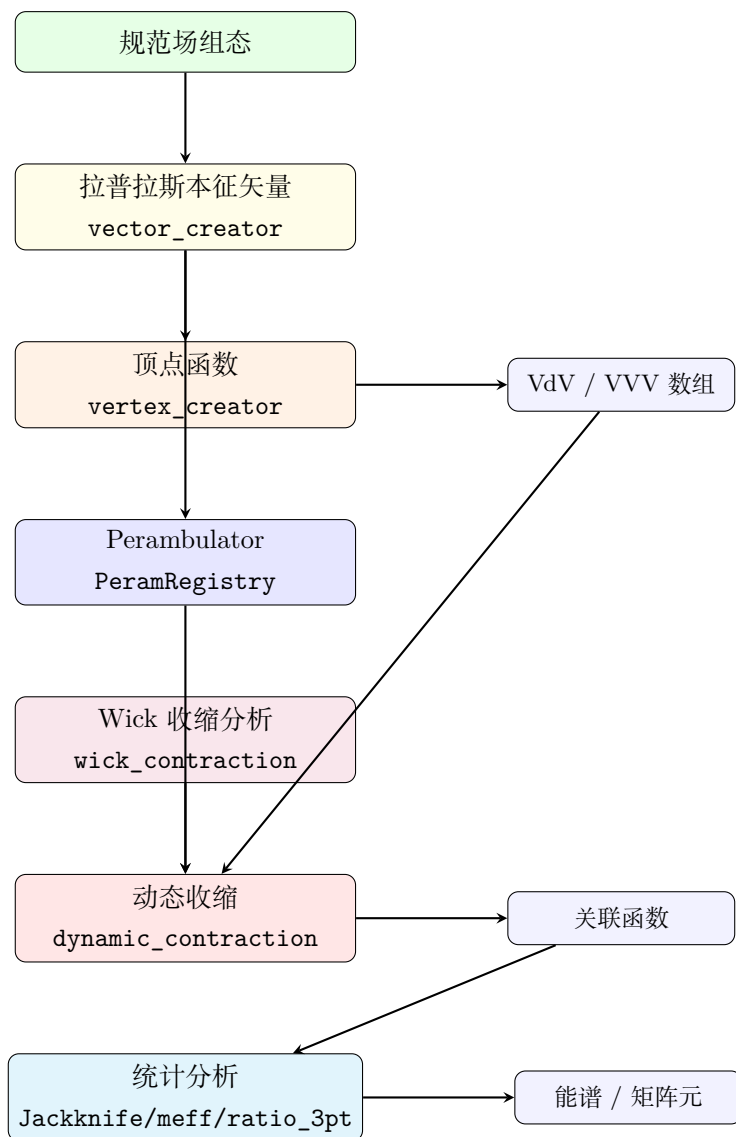


图 1: lqddb 数据流全景

物理量/操作	lqcddb 模块/函数	代码实现关键
夸克传播子 D^{-1}	—	外部输入 (perambulator.npy 文件)
Perambulator $\tau_{kl}^{\alpha\beta}$	PeramRegistry	register(flavor, time_labels, data)
γ_5 -厄米共轭	seq_peram()	$\tilde{\tau} = \gamma_5 \tau^* \gamma_5$
本征矢量 $\xi_{\vec{x}}^{(k)}$	vector_creator	check(), normal(), compress_*
动量投影 $e^{-i\vec{p}\cdot\vec{x}}$	vertex_creator	phase_exp_2pt(), phase_exp_3pt()
VDV 顶角 $V_{kl}(\vec{p})$	vertex_creator	Mom_VdV_sink_t(), VdV_sink_t_link()
VVV 顶角 $V_{klm}(\vec{p})$	vertex_creator	Mom_VVV_sink_t()
Ω 蒸馏权重	vertex_creator	create_omega_accelerate()
Wick 收缩枚举	wick_contraction	味分组 $\rightarrow$ 完美匹配 $\rightarrow$ 笛卡尔积 $\rightarrow$ 费米子符号计算
等价图识别	identify_equivalent_diagrams	HybridGraphs (BFS 符号传播)
收缩图可视化	plot_figure_wick	matplotlib 自动布局
einsum 收缩	dynamic_contract	Registry $\rightarrow$ Wick 分析 $\rightarrow$ 校验 $\rightarrow$ cached_contract
收缩路径优化	cached_contract()	FLOP 最小的 opt_einsum 策略
Roofline 带宽分析	analyze_bandwidth	硬件规格 + 切分建议
Jackknife 重采样	Jackknife()	$\bar{x}_{(k)} = -(x - \Sigma)/(N - 1)$
Bootstrap 重采样	Bootstrap()	有放回随机抽样
有效质量 E_{eff}	meff()	log, cosh, GEVP 三种方法
三点/两点比值 R	ratio_3pt()	含 link 折叠、1D/2D 模式
GEVP 求解	solve_gevp()	scipy.linalg.eigh 求解
源时间平均	loop_tsrc()	周期/反周期边界条件
非连通图	dis_connect()	$C_{3\text{pt}}^{\text{disc}}$ 公式
SU(3) 色因子	levi_civita_tensor	$\epsilon^a(3)$
SU(2) CG 系数	SU2combine(), SU2decompose()	CG 系数耦合/分解
γ 矩阵 (DR 基)	gamma(i)	18 种预定义 γ 矩阵
动量 $\rightarrow \gamma$ 映射	PFF_Mom_to_gamma_hewl	Civita 投影

表 2: 物理-代码对应表

索引	表达式	物理含义
0	I	恒等
1-4	$\gamma_1, \gamma_2, \gamma_3, \gamma_4$	矢量 Dirac 矩阵
5	γ_5	手征矩阵
6-8	$-\gamma_i \gamma_4 \gamma_5$	张量-赝标量组合
9-11	$\gamma_i \gamma_4$	动量提升组合
12-15	$\gamma_i \gamma_5$	轴矢量
16	$\frac{1}{2}(\gamma_0 + \gamma_4)\gamma_3\gamma_1$	投影组合
17	$\frac{1}{2}(\gamma_0 - \gamma_4)\gamma_3\gamma_1$	投影组合

表 3: DR 基 γ 矩阵组合

张量	形状	说明
本征矢量	(Nev, Nz, Ny, Nx, Nc)	N_{ev} 个本征矢, 空间 + 颜色
Perambulator	(Nt, Nd, Nd, Nev, Nev)	时间, Dirac 源, Dirac 汇, 本征矢指标
规范场	(Nd-1, Nt, Nz, Ny, Nx, Nc, Nc)	3 个空间方向, 时间, 空间, SU(3) 色
VDV 顶角	(N_mom, Nev, Nev)	动量, 本征矢源, 本征矢汇
VVV 顶角	(N_mom, Nev, Nev, Nev)	动量, 三个本征矢指标
γ 矩阵	(4, 4)	Dirac 自旋空间
投影算符	(4, 4)	重子自旋投影
动量列表	[pz, py, px]	z 分量最快

表 4: lqcddb 张量约定